



BLUE BITS

البرمجة التفرعية

نظري (2)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
كشياء الجديدة - يقرق يشارى سالى الدالى

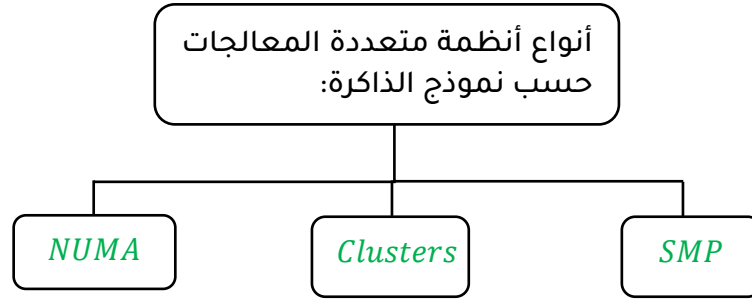
[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com



التنظيمات متعددة المعالجات:

إن الطريقة التقليدية لزيادة أداء النظام هي استعمال معالجات متعددة يمكن أن تعمل على التوازي لتقبل أعباء معينة. إن التنظيمين متعددي المعالجات الأكثر شيوعاً هما: تنظيم المعالجات المتناظرة المتعددة (*symmetric Multiprocessors, SMP*) وتنظيم العناقيد (*Clusters*). ظهرت حديثاً نظم تجارية، تستخدم النفاذ اللامنتظم إلى الذاكرة (*Nonuniform Memory Access, NUMA*).



الأنظمة متعددة المعالجات:

يتألف التنظيم *SMP* من معالجات متماثلة ضمن نفس الحاسوب، تتصل ببعضها البعض بواسطة مسرى أو بنوع ما من شبكة الابدال، إن المشكلة الأساسية التي يجب مواجهتها في تنظيم *SMP* هي في التعامل مع البيانات، فكل معالج له ذاكرته الخاصة به، ولذلك يمكن أن يخزن مجموعة من المعطيات عدة مرات في أكثر من مكان، فإذا أخذنا مجموعة من المعطيات هذه في إحدى الخابيات، يكون لدى أحدها نسخة من الذاكرة الرئيسية والخابية الأخرى نسخة غير صالحة من تلك المعطيات، وقد صُممت بروتوكولات اتساق الخابيات للتغلب على هذه المشكلة.

إن العنقود هو مجموعة من الحواسيب الكاملة الموصولة بينياً يعمل مع بعض كمورد حوسبة موحد، فيبدو كما لو كان حاسوباً واحداً، ويشير التعبير "حاسوب كامل" إلى نظام يستطيع أن يعمل مستقلاً عن العنقود.

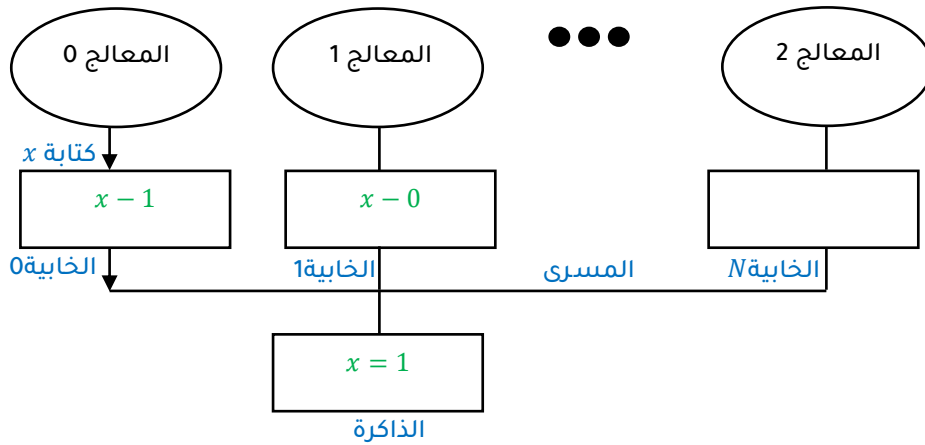
إن نظام النفاذ غير المنتظم إلى الذاكرة *NUMA* هو نظام متعدد المعالجات ذو ذاكرة مشتركة يتغير فيه زمن نفاذ معالج إلى كلمة في الذاكرة بحسب موقع كلمة الذاكرة.

الخابيات caches في البنى المتوازية ذات المعالجات المتعددة والذاكرة المشتركة:

تساهم الخابيات *caches* في البنى المتوازية ذات المعالجات المتعددة والذاكرة المشتركة في رفع أداء البنى بتقليص الزمن اللازم لأي معالج للنفاذ إلى الذاكرة، وإنقاص متطلبات عرض الحزمة على مستوى الذاكرة المحلية وخطوط التوصيل البيني في تلك البنى.

تتسبب تخبئة المعلومات محلياً في مشكلة في الاتساق (أي التوثق من أن جميع المعالجات تتعامل مع القيم ذاتها للمعطيات المخبأة محلياً)، لضمان تنفيذ البرامج تنفيذاً صحيحاً.

تحل هذه المشكلة في المعالجات عتادياً بإنجاز آليات محددة تسمى بروتوكولات اتساق (*cache coherence protocols*)



x بقيم مختلفة في الخابيات وهي قد تكون مختلفة عن القيمة المخزنة في الذاكرة المشتركة.

المعالجات المتعددة المتناظرة *Symmetric Multiprocessors* :

يمكن تعريف *SMP* على أنه نظام حاسوب قائم بذاته له الميزات التالية:

1. يضم معالجين متماثلين بإمكانات متقاربة، أو أكثر.
2. تشترك هذه المعالجات بنفس الذاكرة الرئيسية ووسائل الإدخال/الإخراج (*I/O*) ، وهي موصولة بينياً بواسطة مسرى أو مخطط توصيل داخلي آخر، بحيث يكون زمن نفاذ أي معالج إلى الذاكرة متساوياً تقريباً.
3. تشترك جميع المعالجات بالنفاذ إلى أجهزة الإدخال/الإخراج (*I/O*) عبر القنوات أو ذاتها باستخدام قنوات مختلفة تسمح بالمرور إلى الجهاز نفسه.
4. تستطيع جميع المعالجات أن تنفذ الوظائف ذاتها (ومنه التعبير متناظر).
5. يجري التحكم في النظام بواسطة نظام تشغيل متكامل يتحكم بالتفاعل بين المعالجات وبين برامجها على أصعدة العمل والمهمات، والملفات وعناصر المعطيات.

تضمن *SMP* وحدات المعالجة المركزية (*CPU*) المتعددة في نظام واحد. متماثل (*Symmetric*) أي جميع وحدات المعالجة المركزية في النظام هي نفسها من حيث المعمارية وسرعة الساعة مثلاً كما توجد في نظام *SMP* معالجات متعددة تشترك في جميع موارد النظام الأخرى مثل الذاكرة والقرص الصلب وغير ذلك.

مميزات نظام *SMP* بالمقارنة مع بنية أحادي المعالج:





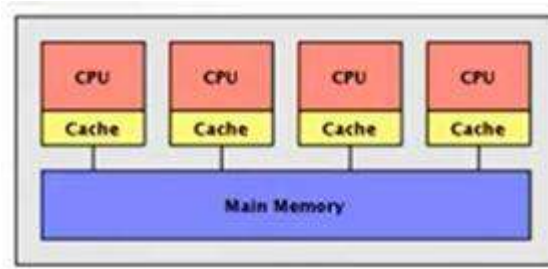
- 1- الأداء (*Performance*): إذا كان بالإمكان تنظيم العمل المطلوب من الحاسوب بحيث تنفذ أجزاء منه تنفيذاً متوازياً، فإن نظاماً متعدد المعالجات سيكون له أداء أفضل من نظام وحيد المعالج من نفس النوع.
- 2- الإتاحة (*Availability*): في نظام معالجات متعددة متناظرة، لن يؤدي عطل معالج ما إلى إيقاف النظام، إذ إن جميع المعالجات يمكن أن تؤدي نفس الوظائف، فيتابع النظام عمله بأداء منخفض.
- 3- نمو تزايد (*Incremental Growth*): يمكن للمستخدم أن يعزز أداء النظام بإضافة معالج إضافي.
- 4- ضبط الأبعاد (*Scaling*): يمكن للبائعين أن يقدموا أنواعاً متنوعة من المنتجات بمميزات متفاوتة من حيث الأداء والثمن بحسب عدد المعالجات المركبة بالنظام.

هناك ميزة جذابة لنظام (*SMP*) وهي أن استخدام معالجات متعددة أمر شفاف بالنسبة للمستخدم إذ يتولى نظام التشغيل جدولة الإجراءات والنياسب على المعالجات المنفردة ويحقق التزامن بينها.

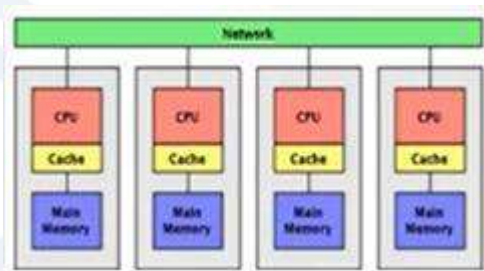
أنماط الحواسيب التفرعية *Types of parallel Computers*:

هناك نمطان مختلفان للحواسيب التفرعية:

- 1- ذاكرة مشتركة، عدة معالجات (*shared memory multiprocessor*) (النمط الذي سندرسه)



- 2- ذاكرة موزعة، لعدة حواسيب (*Distributed memory multicomputer*) (هذا النمط غير مطلوب بالمقرر)

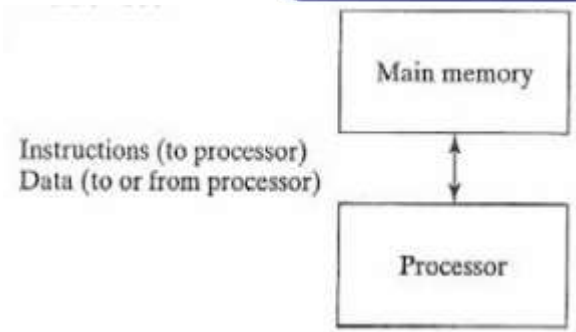


الحاسوب الكلاسيكي *Conventional Computer* (مثلاً)

يتكون من معالج واحد ينفذ برنامج واحد موضوع على ذاكرة وحيدة حيث فيه:

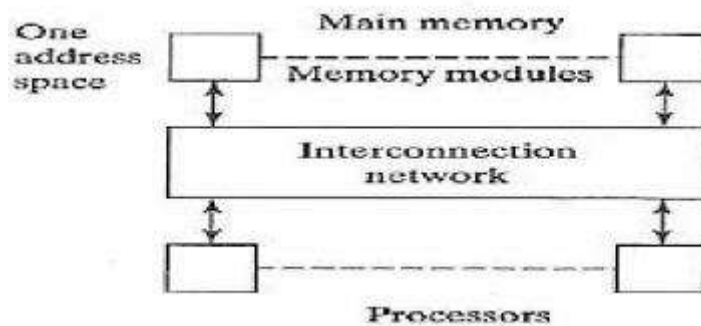
- 1) كل موقع ذاكرة رئيسية يُشار إليه عن طريق عنوانه الخاص.
- 2) العناوين تبدأ من الصفر إلى $2^b - 1$

حيث b تمثل عدد بتات العنوان



نمط الذاكرة مشتركة، عدة معالجات "Shared Memory Multiprocessor System"

- لنفرض أننا أردنا تطبيق هذا المفهوم انطلاقاً من الحاسوب الكلاسيكي، فالطريقة الطبيعية لذلك هي توسيع نموذج المعالج الواحد إلى استخدام عدة معالجات كل منها متصل بعدة وحدات ذاكرة (بحيث يستطيع كل معالج الوصول إلى أي وحدة ذاكرة). فيما يُعرف بتكوين الذاكرة المشتركة *shared memory* ، وفيه يكون التواصل بين المعالجات والذاكرة عن طريق ما يُعرف بشبكة الاتصال الداخلي *interconnection network*.



- يتضمن هذا النموذج مساحة عنوان فريدة (يعني أن كل عنوان في الذاكرة الرئيسية هو عنوان فريد تستخدمه جميع المعالجات للوصول إلى المحتوى ضمنه).
- برمجة نظام الذاكرة مشتركة، عدة معالجات:
 - نحتاج ضمان وجود الكود التنفيذي مخزناً في الذاكرة المشتركة، ليقوم كل معالج بتنفيذه.
 - كما سيتم تخزين البيانات لكل برنامج في الذاكرة المشتركة، ويمكن لكل برنامج الوصول لجميع البيانات عند الحاجة.

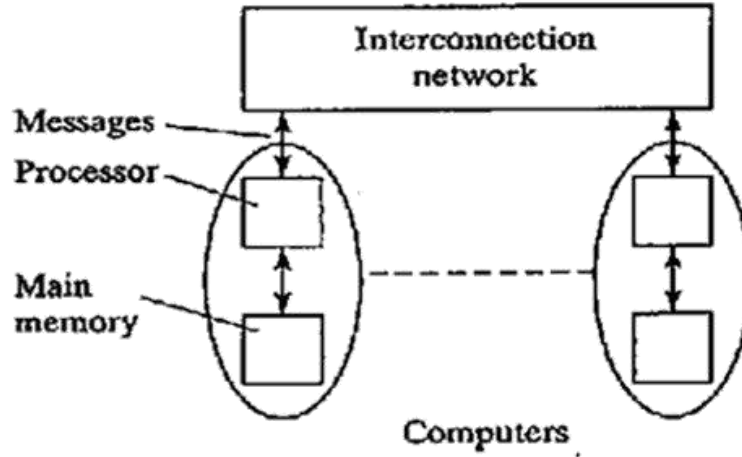
ملاحظة: من إحدى الطرق المستخدمة لتمكين البرمجة التفرعية من إنتاج الكود التنفيذي لكل معالج هي استخدام لغة برمجة عالية المستوى مخصصة للبرمجة التفرعية، وتحتوي على تراكيب وتعليمات خاصة للتصريح عن المتغيرات المشتركة وأقسام الكود التفرعي.

✚ من الطرق التي تدعم مفهوم التفرعية في لغات البرمجة:

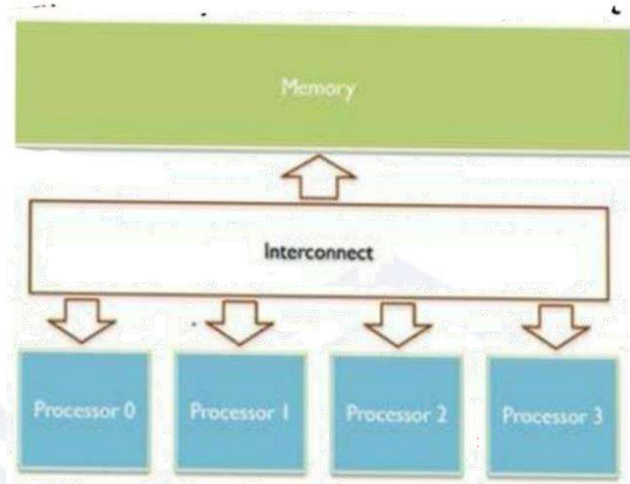
- ✓ استخدام لغة تسلسلية مع توجيهات خاصة للتفرع، مثل: *OpenMP*
- ✓ استخدام ثريدز لتنفيذ أجزاء من الكود على عدة معالجات مختلفة.
- ✓ تعديل صياغة اللغة نفسها لتصبح قابلة لدعم التفرعية، مثل: *UPC*

Message – Passing Multicomputer

حواسيب كاملة متصلة من خلال شبكة الربط البيني *Interconnection Network*



- يتكون كل حاسوب من معالج وذاكرة محلية ولكن هذه الذاكرة غير قابلة للوصول بواسطة المعالجات الأخرى.
- توفر شبكة الربط البيني للمعالجات إرسال رسالة إلى المعالجات الأخرى.
- تحمل الرسالة بيانات من معالج واحد إلى آخر كما هو محدد من البرنامج.



ملاحظة:

كلما زادت استقلالية المعالجات كان زمن تبادل البيانات أقل.

يمكن برمجة Message – Passing Multicomputer

- 1- تصميم لغة برمجة موازية خاصة
- 2- توسيع *Syntax*/الكلمات المحجوزة في لغة برمجة عالية المستوى متتابعة (مستمرة) موجودة للتعامل مع المراسلات.
- 3- استخدام لغة برمجة عالية المستوى متتابعة موجودة و توفير مكتبة من الإجراءات الخارجية للمراسلات.

- سنستخدم في برمجة *Message – Passing* (تبادل الرسائل) لغة برمجة عالية المستوى مثل *C* مع تعزيزها باستدعاءات مكتبة المراسلات *Message – Passing* التي تنفذ المراسلة المباشرة *Process to Process*

في هذه الطريقة من الضروري تحديد ما يلي بشكل صريح:

- 1- العمليات التي يجب تنفيذها "*What*"
- 2- متى يتم تحرير الرسائل بين العمليات المتزامنة "*When*"



3- ما يجب تحريره في الرسائل "What"

- في هذا النوع من برمجة نظام المراسلة نحتاج إلى:
 - 1- طريقة لإنشاء عمليات منفصلة للتنفيذ على أجهزة الحاسوب المختلفة.
 - 2- طريقة لإرسال واستقبال الرسائل.

هناك طريقتان لإنشاء العمليات:

- 1- *Static Process Creation* إنشاء العمليات الثابتة
- 2- *Dynamic Process Creation* إنشاء العمليات المتغيرة الديناميكية

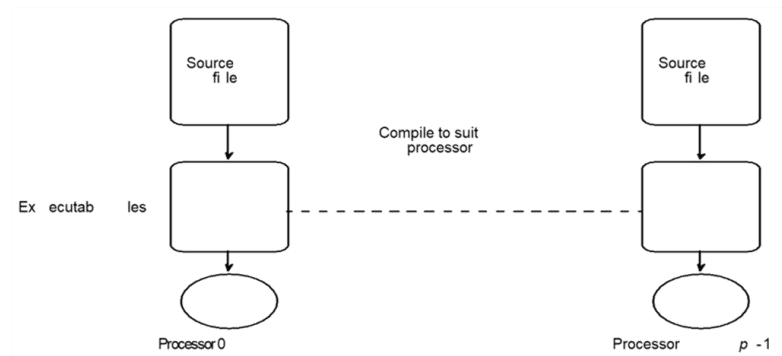
Multiple Program, Multiple Data (MPMD) Model

برامج متعددة، بيانات متعددة:

تتم كتابة برامج مختلفة لكل معالج، بحيث يكفي وجود برنامجين فقط

* A Master Program

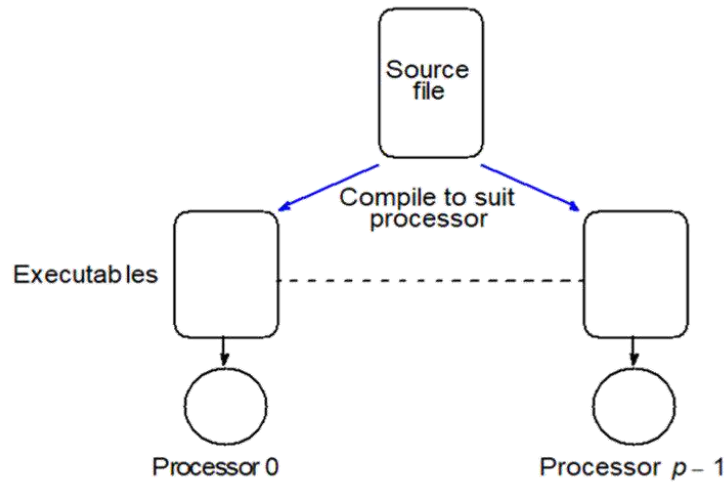
* A Slave Program



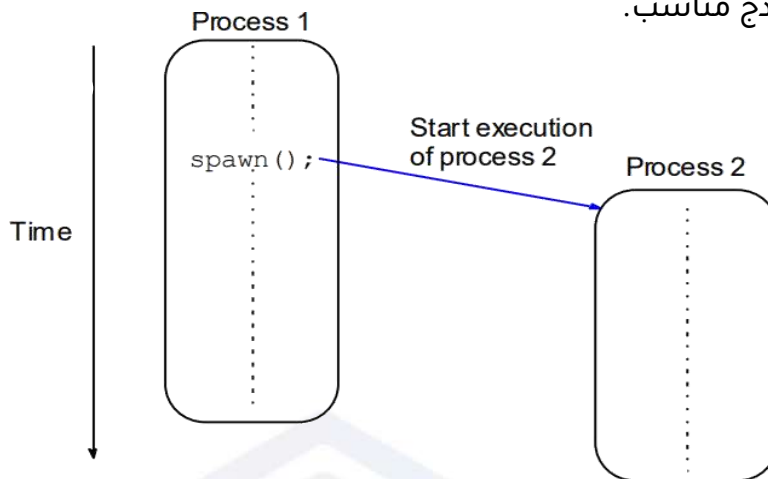
Single Program Multiple Data (SPMD) Model

برنامج وحيد، بيانات متعددة

- إنشاء العمليات الثابتة *Static*
 - 1- يعد النموذج *SPMD* نموذج مناسب
 - 2- العمليات المختلفة تم دمجها في برنامج واحد
 - 3- تحدد *control statement* أجزاء مختلفة لتنفيذها على كل معالج
 - 4- بدء تشغيل جميع الملفات التنفيذية معاً
 - 5- إنه النهج الرئيسي لبروتوكول الرسائل الفعالة *MPI*



- إنشاء العمليات المتغيرة الديناميكية *dynamic*
 - 1- قد يتم كتابة برنامجين مختلفين، برنامج رئيسي وبرنامج مخدم.
 - 2- يعد نموذج *MPMD* نموذج مناسب.



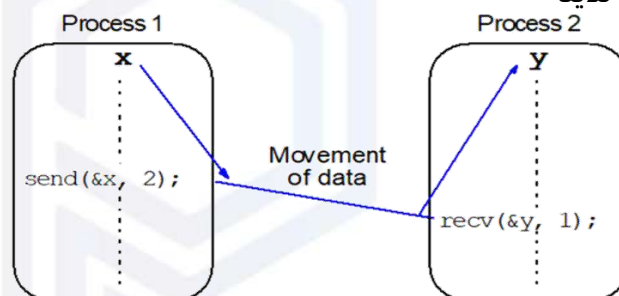
Basic Point to Point Send and Receive Routinen

غالباً ما يكون لاستدعاءات مكتبة *Message – Passing* الشكل التالي:

- تابع إرسال *Send (Parameter – List)*
- تابع تلقي *Recv (Parameter – List)*

من أجل لغة برمجة *C* لدينا

- *Send (&X, destination – id);*
- *Recv (&Y, Source_id);*

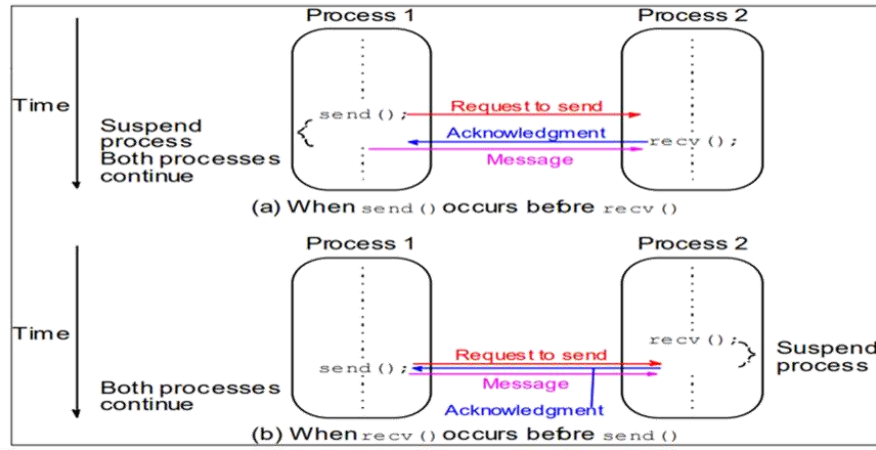


Synchronous Message Passing

رسالة متزامنة:

- مجموعة الإجراءات الروتينية التي يتم إرجاعها عند اكتمال نقل الرسائل
- 1- روتين الإرسال المتزامن: ينتظر حتى يتم قبول الرسالة الكاملة بواسطة عملية الاستلام قبل العودة.
- 2- روتين الاستلام المتزامن: ينتظر حتى تصل الرسالة المتوقعة.
- تقوم الإجراءات الروتينية بتنفيذ إجرائيتين:
 - 1- ينقلون بيانات
 - 2- يقومون بمزامنة العمليات
- لا تحتاج عمليات الإرسال والاستلام المتزامنة إلى تخزين المخزن المؤقت للرسائل، تستخدم هذه الإجراءات الروتينية بروتوكول ثلاثي التوجه.

Synchronous send() and recv() using three-way protocol



Asynchronous Message Passing

- الإجراءات الروتينية التي لا تنتظر اكتمال الإجراءات قبل العودة
- عادة ما تتطلب التخزين المحلي للرسائل

ملاحظة:

روتينيات تبادل الرسائل تقسم إلى نوعين متزامنة *Synchronous* وغير متزامنة *Asynchronous*

Asynchronous Routines changing to Synchronous Routines

تغيير الإجراءات غير المتزامنة إلى إجراءات متزامنة

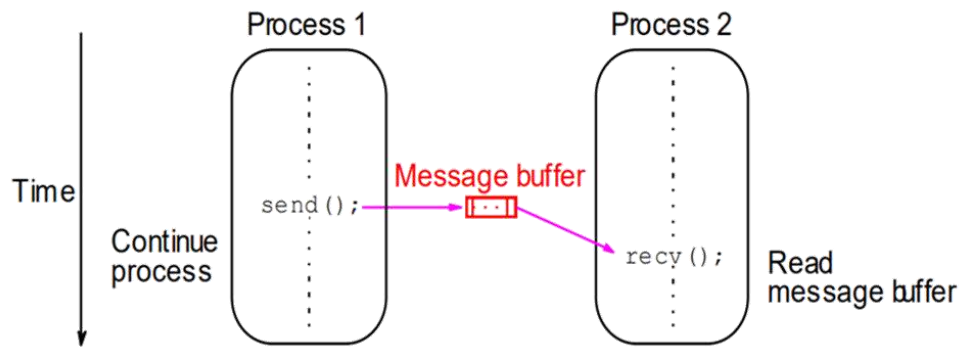
- 1- عندما تكون المخازن المؤقتة محدودة الطول ونريد الوصول إلى نقطة، يتم تعليق روتين الإرسال بسبب استنفاد جميع مساحة التخزين المؤقتة المتاحة
 - 2- بعد ذلك سينتظر روتين الإرسال حتى يصبح التخزين متاحاً مرة أخرى.
- أي بعد ذلك يتصرف الروتين كروتين متزامن *Synchronous*

MPI Definitions of Block and Non – Blocking

Blocking العودة بعد اكتمال الإجراءات المحلية بشكل عام على الرغم من أن نقل الرسالة لم يتم الانتهاء منها

Non – Blocking العودة على الفور

يفترض أن تخزين البيانات المستخدم للنقل لم يتم تعديله بواسطة البيانات اللاحقة قبل استخدامها للنقل ويترك للمبرمج التأكد من ذلك



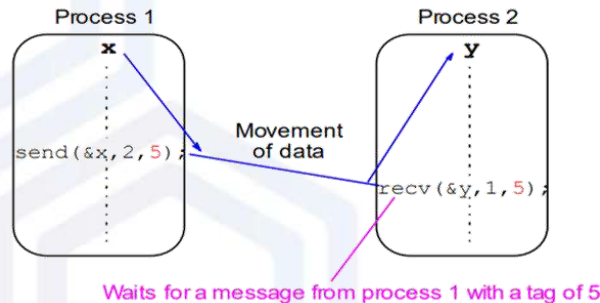
Message Tag

- يستخدم للتمييز بين أنواع مختلفة من الرسائل التي يتم إرسالها.
- يتم حمل علامة الرسالة داخل الرسالة
- إذا لم تكن مطابقة النوع الخاص مطلوبة يتم استخدام علامة رسالة حرف بدل من *Wild Card* ، لذلك إن *recv()* سوف يتطابق مع أي *send()*

مثال:

لإرسال رسالة x مع $message\ tag = 5$ من المصدر 1 إلى الوجهة 2 وتلقي y

To send a message, x , with message tag 5 from a source process, 1, to a destination process, 2, and assign to y :





MPI: Message Passing Interface

- مكتبة تبادل الرسائل *Message Passing Library* معيار تم تطويره من قبل مجموعة من الأكاديميين والشركاء الصناعيين لتعزيز الاستخدام على نطاق أوسع والقدرة على النقل
- *MPI* تملك عدد كبير من الروتينيات (أكثر من 120)
- جميع روتينيات *MPI* تبدأ بالبادئة *MPI_* والحرف بعدها حرف كبير

MPI: Process Creation and Execution

قبل أي استدعاء لتوابع *MPI* يجب تهيئة الكود باستخدام *MPI_Init()* وبعد جميع الاستدعاءات يجب ان ينتهي الكود باستخدام *MPI_Finalize()*

```
1  main(int argc, char * argv[])
2  {
3      MPI_Init(&argc, &argv);
4      ...
5      MPI_Finalize();
6  }
```

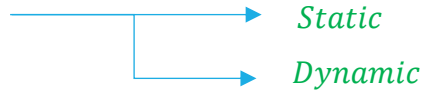
Communicators حوض الإتصال

- في البداية تم تسجيل جميع العمليات في عالم يسمى *MPI_COMM_WORLD* و تم منح كل عملية مرتبة فريدة *Unique Rank* و هي رقم يتراوح من 0 و حتى $P - 1$ من أجل P من العمليات.
- *MPI_COMM_WORLD* هو حوض اتصال يحدد نطاق عملية اتصال.
- العمليات لها رتب مرتبطة بالمتصلين
- يمكن إنشاء أحواض اتصال أخرى لمجموعة من العمليات



ملخص المحاضرة الثانية

Process Creation



كتابة البرامج:

MPMD

- برامج متعددة و بيانات
- مفيد في إنشاء عمليات *dynamic*

SPMD

- برنامج وحيد و بيانات متعددة
- مفيد في إنشاء عملية *Static*

روتينيات تبادل الرسائل

Synchronous

- ↳ متزامنة
- ↳ تنتظر اكتمال إجراءات نقل الرسائل قبل العودة
- ↳ لا تحتاج عمليات التبادل إلى تخزين مؤقت للرسائل

Asynchronous

- ↳ غير متزامنة
- ↳ لا تنتظر اكتمال إجراءات نقل الرسائل قبل العودة
- ↳ تتطلب التخزين المحلي للرسائل

MPI Definitions

Blocking

العودة بعد اكتمال الإجراءات بشكل تام على الرغم من أن نقل الرسالة لم يتم الانتهاء منها

Non – Blocking

العودة على الفور

2026/2025

السنة الرابعة



BLUE BITS



أعضاء الفريق



الفريق التقني



الفريق العلمي

إبراهيم هبرة	عائشة العُمر العلّوش	إيناس بيلوني	ظلال بابا
إسراء حاج حمدي	عمر قظان	تسنيم طالب	عائشة قلّه
بتول لبّان	فاطمة الحسين	حلا إبراهيم	عائشة مُصطفى
بشرى كوكش	قاسم يحيى	دلال تسقيه	قمر حَمَد
بنيامين حكمان	كريست ياغجيان	ديانا صَبّاغ	محمد يمان جلال
تسنيم حوكان	محمد منير	زهير حَقّامي	محمود غزال
تسنيم نور الدّين	مديحة جَمّال	شكرية سُلطان	وفاء حَقّامي
جميل هلال	ميرا فضلي	شهد كرّوم	يوسف نيّال
جودي جلال	نجم الدّين درويش		
خديجة بيازيد	هبة الله كتلو		
رشا معراوي	وداد قلعيّة		
سدرة مبيض	ياسر صَبّاغ		

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

